

SESSION 08

Dictionary in Python

What is a Dictionary in Python?

A dictionary in Python is an unordered collection of data values used to store data in key-value pairs. Think of it like a real-world dictionary where you look up a word (the key) to find its definition (the value), or a phone book where a name (key) leads you to a phone number (value).

```
# List — just values, accessed by position
marks = [85, 90, 78]
print(marks[0]) # 85 — but what does 85 stand for?

# Dictionary — values labelled with meaningful keys
marks = {"math": 85, "science": 90, "english": 78}
print(marks["math"]) # 85 — crystal clear what this means
```

Key Properties of a Dictionary

Key-Value Pairs

Every value is stored under a unique, descriptive key, making data retrieval intuitive.

Unordered

Dictionaries don't rely on index position; data is accessed via keys. (Note: Python 3.7+ preserves insertion order.)

Mutable

Dictionaries are dynamic; you can add, change, or remove key-value pairs after creation.

Unique Keys

Each key within a dictionary must be unique. Duplicate keys will simply overwrite previous entries.

Immutable Keys

Keys must be of an immutable data type (e.g., strings, numbers, tuples). Lists and other mutable objects cannot be used as keys.

Creating a Dictionary

Dictionaries in Python are created using curly braces `{}` with key-value pairs. Each pair consists of a key followed by a colon `:`, and then its associated value. These pairs are separated by commas.

```
# Empty dictionary
empty_dict = {}
print(empty_dict) # Output: {}

# Dictionary with string keys
student = {
    "name": "Rahul",
    "age": 21,
    "city": "Delhi",
    "is_active": True
}
print(student)
# Output: {'name': 'Rahul', 'age': 21, 'city': 'Delhi', 'is_active': True}

# Dictionary with number keys
roll_to_name = {
    101: "Priya",
    102: "Karan",
    103: "Sana"
}
print(roll_to_name)
# Output: {101: 'Priya', 102: 'Karan', 103: 'Sana'}

# Values can be any type — including lists and other dictionaries
profile = {
    "name": "Aman",
    "marks": [85, 90, 78],
    "address": {"city": "Pune", "pin": 411001}
}
print(profile)
# Output: {'name': 'Aman', 'marks': [85, 90, 78], 'address': {'city': 'Pune', 'pin': 411001}}
```

You can check the type and length of a dictionary:

```
print(type(student)) # Output: <class 'dict'>
print(len(student))  # Output: 4
```

 **Watch Out:** Keys must be unique. If you repeat a key when creating or updating a dictionary, the new value associated with that key will simply overwrite the old one. Python will **not** raise an error for this operation.

```
my_dict = {"a": 1, "b": 2, "a": 3}
print(my_dict) # Output: {'a': 3, 'b': 2} - the first 'a' was overwritten
```

Accessing Values

Python offers two main ways to access dictionary values: square brackets `[]` and the `get()` method. Their behaviour differs significantly when a key is not found.

Using Square Brackets `[]`

Directly access values using `[]`. If the key doesn't exist, this method will raise a `KeyError`, halting program execution.

```
student = {"name": "Rahul", "age": 21, "city": "Delhi"}

# Accessing existing keys
print(student["name"])    # Output: Rahul
print(student["age"])     # Output: 21

# Accessing a non-existent key will cause a KeyError
# print(student["marks"]) # Uncommenting this line causes: KeyError: 'marks'
```

Using the `get()` Method

The `get()` method offers a safer alternative. If the key is not found, it returns `None` by default, preventing a crash. You can also specify a custom default value.

```
student = {"name": "Rahul", "age": 21, "city": "Delhi"}

# Accessing existing keys
print(student.get("name")) # Output: Rahul

# Key not found: returns None (no crash)
print(student.get("marks")) # Output: None

# Key not found: returns a specified default value
print(student.get("marks", 0)) # Output: 0
print(student.get("grade", "N/A")) # Output: N/A
```

📌 **Quick Tip:** Use `[]` when you are sure the key exists. Opt for `get()` when the key might be absent, to handle missing data gracefully and avoid program crashes.

Adding and Updating Values

Adding new key-value pairs or updating existing ones in a dictionary uses the same intuitive syntax: `dictionary[key] = value`. Python handles the logic internally: if the `key` exists, its value is updated; otherwise, a new pair is created.

```
student = {"name": "Rahul", "age": 21}

# Adding new key-value pairs
student["city"] = "Delhi"
student["marks"] = 85
print(student)
# Output: {'name': 'Rahul', 'age': 21, 'city': 'Delhi', 'marks': 85}

# Updating an existing value
student["age"] = 22
student["city"] = "Mumbai"
print(student)
# Output: {'name': 'Rahul', 'age': 22, 'city': 'Mumbai', 'marks': 85}
```

 **Quick Tip:** Python does not differentiate between "add" and "update" operations for dictionary entries. It simply assigns the specified `value` to the given `key`, regardless of whether that key existed previously or not.

Removing Items

Python provides flexible methods to remove key-value pairs or clear a dictionary entirely. Understanding their differences helps in robust error handling.



pop(key)

Removes and returns the value for a specified key. It can also return a default value if the key is not found, preventing errors.



del Statement

Deletes a specific key-value pair. If the key does not exist, it will raise a `KeyError`, halting program execution.



clear() Method

Removes all key-value pairs from the dictionary, leaving it empty. This method does not take any arguments and returns `None`.

```
student = {"name": "Rahul", "age": 21, "city": "Delhi", "marks": 85}

# pop(key) — removes and returns the value
removed = student.pop("marks")
print(f"Removed value: {removed}") # Output: Removed value: 85
print(f"Dictionary after pop: {student}")
# Output: Dictionary after pop: {'name': 'Rahul', 'age': 21, 'city': 'Delhi'}

# pop with default — no crash if key doesn't exist
value = student.pop("grade", "Not Found")
print(f"Attempt to pop non-existent key: {value}") # Output: Attempt to pop non-existent key: Not Found

# del — removes without returning
del student["city"]
print(f"Dictionary after del: {student}")
# Output: Dictionary after del: {'name': 'Rahul', 'age': 21}

# clear() — removes all items
student.clear()
print(f"Dictionary after clear: {student}") # Output: Dictionary after clear: {}
```

⚠ **Watch Out:** Using `del dictionary[key]` will raise a `KeyError` if the key does not exist. For safer operations when a key might be absent, `pop()` with a default value is highly recommended.

Checking if a Key Exists

Efficiently determine if a key is present in a dictionary using the `in` and `not in` operators. This approach helps prevent `KeyError` exceptions when attempting to access or modify dictionary entries.

```
student = {"name": "Rahul", "age": 21, "city": "Delhi"}
```

```
# Checking if a key exists
```

```
print("name" in student)    # Output: True
```

```
print("marks" in student)   # Output: False
```

```
print("grade" not in student) # Output: True
```

Practical Applications

Leverage these operators to safely access values or to conditionally add new entries without accidentally overwriting existing ones.

```
# Check before accessing
```

```
if "marks" in student:
```

```
    print(f"Marks: {student['marks']}")
```

```
else:
```

```
    print("Marks not recorded yet.")
```

```
# Output: Marks not recorded yet.
```

```
# Check before adding — avoid overwriting
```

```
if "city" not in student:
```

```
    student["city"] = "Unknown" # This condition is false, 'city' already exists
```

```
print(student)
```

```
# Output: {'name': 'Rahul', 'age': 21, 'city': 'Delhi'}
```

 **Important:** The `in` operator checks for keys, not values. To check if a value exists within a dictionary, you would need to iterate through its values or use `value in dictionary.values()`.

Dictionary Methods (Part 1)

Python dictionaries offer several built-in methods to efficiently interact with their contents. The `keys()`, `values()`, and `items()` methods are particularly useful for accessing different components of the dictionary.



`keys()`

Returns a **view object** that displays a list of all the keys in the dictionary. This view object is dynamic, meaning it reflects any changes made to the dictionary.



`values()`

Returns a **view object** that displays a list of all the values in the dictionary. Like `keys()`, this view updates dynamically when the dictionary changes.



`items()`

Returns a **view object** that displays a list of a dictionary's key-value pairs as **tuples**. This combined view also updates dynamically with dictionary modifications.

```
student = {"name": "Rahul", "age": 21, "city": "Delhi"}

# keys() — returns a view of all keys
print(student.keys())
# Output: dict_keys(['name', 'age', 'city'])

# values() — returns a view of all values
print(student.values())
# Output: dict_values(['Rahul', 21, 'Delhi'])

# items() — returns a view of all key-value pairs as tuples
print(student.items())
# Output: dict_items([('name', 'Rahul'), ('age', 21), ('city', 'Delhi')])

# You can convert these views to a list if you need indexing or other list operations
keys_list = list(student.keys())
print(keys_list[0]) # Output: name
```

 **Quick Tip:** The view objects returned by `keys()`, `values()`, and `items()` are iterable. They are most commonly used in loops to process dictionary contents, a topic we'll explore in detail later.

Dictionary Methods (Part 2)

Building on basic operations, Python provides additional methods like `update()` for merging, `get()` for safe access, and `copy()` for creating independent duplicates. These methods are crucial for writing robust and efficient dictionary manipulations.



`update()`

Adds or updates multiple key-value pairs from another dictionary or iterable, making it efficient for bulk changes.



`get()`

Offers a safer way to access values by key. It returns `None` or a specified default value if the key isn't found, preventing `KeyError`.



`copy()`

Creates a shallow copy of the dictionary. This is vital when you want to modify a dictionary without affecting the original.

```
student = {"name": "Rahul", "age": 21}
```

```
# update() — add or update multiple pairs at once
```

```
student.update({"city": "Delhi", "age": 22})
```

```
print(student)
```

```
# Output: {"name": "Rahul", "age": 22, "city": "Delhi"}
```

```
# If the key exists, it gets updated. If not, it gets added.
```

```
# get() — safer access (already covered, quick recap)
```

```
print(student.get("name"))      # Output: Rahul
```

```
print(student.get("marks", 0))  # Output: 0
```

```
# copy() — creates an independent copy
```

```
original = {"a": 1, "b": 2}
```

```
copy = original.copy()
```

```
copy["c"] = 3
```

```
print(original)  # Output: {"a": 1, "b": 2} — unchanged
```

```
print(copy)      # Output: {"a": 1, "b": 2, "c": 3}
```

📌 **Important Note:** Using `copy = original` merely creates a new reference to the same dictionary. Any changes made via `copy` would also alter `original`. The `copy()` method is essential for creating truly independent dictionaries.

Looping Through a Dictionary (Part 1)

Iterating over dictionaries is a fundamental task in Python, allowing you to process their contents efficiently. Python offers several straightforward ways to loop through keys, values, or both simultaneously.



Looping over Keys

By default, iterating over a dictionary directly processes its keys. You can also explicitly use the `.keys()` method for the same effect.



Looping over Values

When you only need the data stored in the dictionary, the `.values()` method provides a direct way to iterate through all the values.



Accessing Keys and Values

While iterating through keys, you can access the corresponding value for each key using standard bracket notation (`dictionary[key]`).

```
student = {"name": "Rahul", "age": 21, "city": "Delhi"}
```

```
# Looping over keys (default behaviour)
```

```
print("--- Looping over Keys ---")
```

```
for key in student:
```

```
    print(key)
```

```
# Output:
```

```
# name
```

```
# age
```

```
# city
```

```
# Explicitly looping over keys (same result)
```

```
for key in student.keys():
```

```
    print(key)
```

```
# Output:
```

```
# name
```

```
# age
```

```
# city
```

```
# Looping over values
```

```
print("\n--- Looping over Values ---")
```

```
for value in student.values():
```

```
    print(value)
```

```
# Output:
```

```
# Rahul
```

```
# 21
```

```
# Delhi
```

```
# Looping over keys and accessing values
```

```
print("\n--- Looping over Keys & Values ---")
```

```
for key in student:
```

```
    print(f"{key}: {student[key]}")
```

```
# Output:
```

```
# name: Rahul
```

```
# age: 21
```

```
# city: Delhi
```

 **Next Up:** For scenarios where you need both the key and value in a more Pythonic way, the `.items()` method is invaluable. We'll explore this and other advanced looping techniques in the next section.

Looping Through a Dictionary (Part 2)

While direct iteration gives you keys and `.values()` gives you, well, values, the `.items()` method is the most Pythonic and efficient way to access both key and value simultaneously. This is especially useful for performing operations that require both pieces of information.



`items()` for Key-Value Pairs

This method returns a view object that displays a list of a dictionary's key-value pairs as tuples. You can easily unpack these tuples directly in a `for` loop.



Filtering and Conditional Logic

With direct access to both key and value, `items()` simplifies filtering dictionaries based on specific criteria for either the key, the value, or both.



Aggregating and Analyzing Data

It's ideal for tasks like counting occurrences, summing values, or building new data structures where each item's key-value context is important.

```
student = {"name": "Rahul", "age": 21, "city": "Delhi"}

# Looping over key-value pairs using items() — the cleanest way
print("--- Looping with items() ---")
for key, value in student.items():
    print(f'{key}: {value}')
# Output:
# name: Rahul
# age: 21
# city: Delhi

# Practical use — filter based on values
marks = {"math": 85, "science": 92, "english": 60, "hindi": 45}

print("\n--- Subject Performance ---")
for subject, score in marks.items():
    if score >= 80:
        print(f'{subject}: Excellent')
    elif score >= 60:
        print(f'{subject}: Good')
    else:
        print(f'{subject}: Needs Improvement')
# Output:
# math: Excellent
# science: Excellent
# english: Good
# hindi: Needs Improvement

# Counting values that match a condition (using .values() directly here)
passed_subjects = 0
for score in marks.values():
    if score >= 60: # Assuming 60 is the passing mark
        passed_subjects += 1
print(f"\nSubjects passed: {passed_subjects}")
# Output: Subjects passed: 3
```

 **Quick Tip:** `items()` is almost always the cleanest way to loop through a dictionary when you need both key and value. Remember this pattern — you will use it constantly.

Practice Questions: Dictionaries

Put your dictionary knowledge to the test with these practical questions. Apply the methods and concepts you've learned to solve common dictionary-related problems.

Q79: Student Profile (HW)

Create a dictionary for a student, including keys like `name`, `age`, `city`, and `marks` (as a list of scores). Print each piece of information using its key.

Q80: Safe Subject Access (HW)

Define a dictionary with five subjects and their respective marks. Utilize the `get()` method to try accessing a subject that is not in the dictionary, ensuring it prints "Not Available" as a default.

Q81: Product Price Lookup (HW)

Construct a dictionary containing four product names and their prices. Prompt the user to enter a product name. Use the `in` keyword to check if it exists; if so, display its price. Otherwise, inform the user "Product not found."

Q82: World Capitals (HW)

Create a dictionary mapping five countries to their capital cities. Iterate through this dictionary using the `items()` method and print each pair in the format: **Country** → **Capital**.

Q83: Subject Performance Analysis

Given a dictionary of marks for different subjects, loop over its `values()` to calculate and print the total marks and the average mark obtained.

Q84: Top Scorers

Populate a dictionary with six student names and their corresponding marks. Loop through it and print the names of all students who achieved a score above 75.

Q85: Dictionary Merge Function

Write a Python function named `merge_dicts(d1, d2)` that accepts two dictionaries (`d1` and `d2`) as arguments and returns a new dictionary formed by merging them using the `update()` method. Ensure `d1` remains unchanged.

Built-in Functions on Dictionaries

Python provides several convenient built-in functions that can be directly applied to dictionaries or their views (keys, values, items) to perform common operations like counting elements, finding totals, or sorting. These functions simplify data manipulation and offer efficient ways to work with dictionary contents.

#

len(): Count Items

Returns the total number of key-value pairs (items) in the dictionary, indicating its size.

Σ

sum(): Aggregate Values

Calculates the sum of all numeric values in a dictionary. When used directly on a dictionary, it sums its keys (if numeric). For values, explicitly use `.values()`.

↑

min() / max(): Find Extremes

Identifies the smallest or largest key in a dictionary by default. To find the min/max value, apply it to `.values()`.

⇅

sorted(): Order Elements

Returns a new sorted list of keys from the dictionary. To get a sorted dictionary by keys or values, further processing with `.items()` and `dict()` is needed.

```
pythonmarks = {"math": 85, "science": 92, "english": 78, "hindi": 60}

# len() — number of key-value pairs
print(len(pythonmarks))      # Output: 4

# sum() on values
print(sum(pythonmarks.values()))  # Output: 315

# min() and max() on values
print(min(pythonmarks.values()))  # Output: 60
print(max(pythonmarks.values()))  # Output: 92

# min() and max() on keys (by default it uses keys, sorted alphabetically)
print(min(pythonmarks))          # Output: english
print(max(pythonmarks))          # Output: science

# sorted() on keys — returns a sorted list of keys
print(sorted(pythonmarks))
# Output: ['english', 'hindi', 'math', 'science']

# Sorted dictionary by keys
sorted_by_keys = dict(sorted(pythonmarks.items()))
print(sorted_by_keys)
# Output: {'english': 78, 'hindi': 60, 'math': 85, 'science': 92}
```

As shown in the examples, `sum()`, `min()`, and `max()` can be applied directly to `.values()` to operate on the dictionary's numeric contents. When used without `.values()`, `min()`, `max()`, and `sorted()` operate on the dictionary's keys.

❗

Note: The `sorted()` function by default sorts dictionary keys. To sort a dictionary by its values or based on custom logic, you'll typically need to use a `lambda` function, a topic we'll cover later after discussing nested dictionaries.

Nested Dictionaries (Part 1: Dict Inside Dict)

A nested dictionary is a powerful data structure in Python where the values of a dictionary are themselves dictionaries. This allows you to store complex, hierarchical data, ideal for representing records like student profiles, product catalogs, or structured API responses.

```
students = {
    "101": {"name": "Rahul", "age": 21, "city": "Delhi"},
    "102": {"name": "Priya", "age": 20, "city": "Mumbai"},
    "103": {"name": "Karan", "age": 22, "city": "Pune"}
}

# Accessing nested values — chain the keys
print(students["101"]["name"]) # Output: Rahul
print(students["102"]["city"]) # Output: Mumbai

# Adding a new student
students["104"] = {"name": "Sana", "age": 19, "city": "Chennai"}

# Updating a nested value
students["101"]["city"] = "Bangalore"

# Looping through a nested dictionary
for roll, info in students.items():
    print(f"Roll {roll}: {info['name']} from {info['city']}")
# Output:
# Roll 101: Rahul from Bangalore
# Roll 102: Priya from Mumbai
# Roll 103: Karan from Pune
# Roll 104: Sana from Chennai
```

As illustrated, you can easily access, add, and update values within a nested dictionary by chaining keys. Iterating through the outer dictionary using `.items()` provides both the key (e.g., student roll number) and its corresponding inner dictionary (e.g., student information).

📌 Important: Nested dictionaries are crucial for handling real-world datasets that often have layered or related information. They allow for organised and intuitive data management.

Nested Dictionaries (Part 2: List Inside Dict)

In addition to dictionaries, lists can also serve as values within another dictionary. This powerful combination allows a single key to store multiple related pieces of information, perfect for representing collections of items associated with a primary entity, like a student's enrolled subjects or a department's staff roster.

```
# Example 1: Student profile with lists for subjects and marks
student = {
    "name": "Rahul",
    "age": 21,
    "subjects": ["Math", "Science", "English"],
    "marks": [85, 92, 78]
}

# Accessing list elements inside a dictionary
print(student["subjects"])    # Output: ['Math', 'Science', 'English']
print(student["subjects"][0]) # Output: Math
print(student["marks"][1])    # Output: 92

# Modifying a list directly within the dictionary
student["subjects"].append("Hindi")
student["marks"].append(70)
print(student)
# Output: {'name': 'Rahul', 'age': 21, 'subjects': ['Math', 'Science', 'English', 'Hindi'], 'marks': [85, 92, 78, 70]}

# Example 2: Real-world scenario - company departments with employee lists
company = {
    "engineering": ["Rahul", "Priya", "Karan"],
    "marketing": ["Sana", "Aman"],
    "hr": ["Neha"]
}

# Looping through department and employee lists
for dept, employees in company.items():
    print(f"{dept.upper()} — {len(employees)} members:")
    for emp in employees:
        print(f"  - {emp}")
# Output:
# ENGINEERING — 3 members:
#   - Rahul
#   - Priya
#   - Karan
# MARKETING — 2 members:
#   - Sana
#   - Aman
# HR — 1 members:
#   - Neha
```

As demonstrated, accessing elements from a list nested within a dictionary is straightforward: first access the dictionary key, then use list indexing. Lists within dictionaries are mutable, meaning you can add, remove, or modify elements directly. This flexibility makes them ideal for managing dynamic collections of data.

Sorting Nested Data with Lambda

When working with complex data structures like lists of dictionaries or dictionaries that need custom sorting, the `sorted()` function becomes incredibly powerful when combined with a lambda function. A lambda function allows you to define a small, anonymous function directly within the `key` argument of `sorted()`, specifying precisely what value to use for comparison.

Sorting a List of Dictionaries

To sort a list of dictionaries by a specific key, we use a lambda function to tell `sorted()` which dictionary element to use as the sorting key:

```
students = [
    {"name": "Rahul", "marks": 85},
    {"name": "Priya", "marks": 92},
    {"name": "Karan", "marks": 78},
    {"name": "Sana", "marks": 95}
]

# Sort by marks (ascending)
by_marks = sorted(students, key=lambda s: s["marks"])
print("Sorted by marks (ascending):")
for s in by_marks:
    print(s)
# Output:
# {'name': 'Karan', 'marks': 78}
# {'name': 'Rahul', 'marks': 85}
# {'name': 'Priya', 'marks': 92}
# {'name': 'Sana', 'marks': 95}

# Sort by marks (descending)
by_marks_desc = sorted(students, key=lambda s: s["marks"], reverse=True)
# print(by_marks_desc) # Output will be reverse of above

# Sort by name (alphabetically)
by_name = sorted(students, key=lambda s: s["name"])
# print(by_name) # Output: [{'name': 'Karan', 'marks': 78}, {'name': 'Priya', 'marks': 92}, ...]
```

Here, `lambda s: s["marks"]` instructs the sort function to use the value associated with the "marks" key for each student dictionary `s` as the sorting criterion.

Sorting a Dictionary by its Values

Dictionaries themselves cannot be directly sorted, but you can get a sorted list of their key-value pairs or create a new ordered dictionary. We achieve this by sorting the `.items()` view, which returns tuples of (key, value):

```
marks = {"math": 85, "science": 92, "english": 60, "hindi": 78}

# Sort by values (ascending)
sorted_marks_items = sorted(marks.items(), key=lambda item: item[1])
sorted_marks_dict = dict(sorted_marks_items)
print("Sorted by values (ascending):")
print(sorted_marks_dict)
# Output: {'english': 60, 'hindi': 78, 'math': 85, 'science': 92}

# Sort by values (descending)
sorted_desc_items = sorted(marks.items(), key=lambda item: item[1], reverse=True)
sorted_desc_dict = dict(sorted_desc_items)
# print(sorted_desc_dict) # Output: {'science': 92, 'math': 85, 'hindi': 78, 'english': 60}
```

 **Quick Tip:** In `lambda item: item[1]`, `item` refers to a (key, value) tuple. So, `item[1]` accesses the value for sorting, while `item[0]` would access the key.

Dictionary Comprehension

Just like list comprehension, dictionary comprehension provides a concise way to create dictionaries in a single line. It streamlines the process of building new dictionaries from existing iterables, making your code more readable and efficient.

```
# Normal way — using a loop
squares = {}
for i in range(1, 6):
    squares[i] = i * i
print(squares)  # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Dictionary comprehension — same result in one line
squares = {i: i * i for i in range(1, 6)}
print(squares)  # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

This example clearly illustrates how a traditional loop to populate a dictionary can be replaced with a single, elegant line of dictionary comprehension, producing the same result.

```
# With a condition
marks = {"math": 85, "science": 92, "english": 60, "hindi": 45}

# Keep only subjects where marks > 80
top = {sub: m for sub, m in marks.items() if m > 80}
print(top)  # {'math': 85, 'science': 92}

# Transform — double all marks
doubled = {sub: m * 2 for sub, m in marks.items()}
print(doubled)  # {'math': 170, 'science': 184, 'english': 120, 'hindi': 90}

# Creating a dict from two lists
subjects = ["math", "science", "english"]
scores = [85, 92, 78]
result = {s: sc for s, sc in zip(subjects, scores)}
print(result)  # {'math': 85, 'science': 92, 'english': 78}
```

Dictionary comprehensions are incredibly versatile, allowing you to filter items based on conditions, transform values, and even combine multiple iterables (like lists) into a new dictionary using functions like `zip()`.

 **Pro Tip:** While powerful, use dictionary comprehensions for concise and readable code. Overly complex expressions might reduce clarity.

Practice Questions: Dictionaries

Test your understanding of Python dictionaries with these practice questions. Focus on applying the concepts we've covered, including creation, manipulation, and advanced techniques.

01

Q86. Subjects and Marks

Create a dictionary of 6 subjects and their respective marks. Print the subject with the highest marks and the one with the lowest, using `max()` and `min()` functions alongside a `lambda` expression.

03

Q88. Student Marks Analysis

Design a dictionary where each key is a student's name and the corresponding value is a list of their marks in 3 different subjects. Calculate and print the total marks and average marks for each student.

02

Q87. Student Details

Create a nested dictionary containing details for 4 students, where each student entry includes their name, age, and city. Write a loop to print the full details of each student in a clear, readable format.

04

Q89. Top 3 Subjects by Marks

Given a dictionary of subjects and their marks, sort it by marks in descending order. Then, print only the top 3 subjects with the highest marks.

Practice Questions: Dictionary

Challenge your understanding of dictionary comprehension with these practical questions. Focus on applying the concise syntax to solve common dictionary manipulation tasks efficiently.

Q90. Numbers to Cubes

Using dictionary comprehension, create a new dictionary where keys are numbers from 1 to 10 (inclusive), and values are the cube of each number.

Q91. Filter Passing Marks

Given an existing dictionary of subjects and their respective marks, use dictionary comprehension to generate a new dictionary that includes only the subjects where the student scored 40 or more (i.e., passed).

Q92. Combine Lists into Dictionary

You have two separate lists: one containing subject names and another containing corresponding marks. Create a dictionary from these two lists using dictionary comprehension, mapping each subject to its mark.